Programming Guidelines

Standards followed in internal projects

Note: these guidelines are not strict obligations — they are strong recommendations intended to keep our codebase consistent and maintainable.

Documentation & Style

- **Documentation:** Require NumPy-style docstrings on all public functions, explicitly including `Parameters`, `Returns`, `Raises`, and a `Notes` section for detailing the rationale behind non-obvious decisions.
- **Type Hinting & Styling:** Adhere to PEP 8 formatting and enforce full type hinting using precise types (e.g., `pd.DataFrame`, `date`, dataclasses) over `Any`, utilizing `from __future__ import annotations`.
- **Automated Linting & Formatting:** Enforce PEP 8 and type hints automatically in your workflow (and CI pipeline) using tools like `ruff` and `mypy` to prevent style and type regressions before code review.

Software Design

- **Software Design:** Avoid premature abstraction by writing explicit implementations (e.g., defining directional corridors directly rather than collapsing them into generic objects).
- **Structured Configuration:** Use structured configuration parsing (e.g., `pydantic-settings` or dataclasses) to validate the environment variables loaded from the `.env` file, ensuring type safety for external configurations and failing fast if an API key is missing.
- **Error Handling:** Define and raise custom, domain-specific exception classes for expected failure states (e.g., `DataFetchError`, `StaleDataError`) instead of catching or raising broad `Exception` base classes.

Correctness & Testing

- **Invariant Checking:** Implement sanity checks as code rather than comments, using `assert` statements that fail loudly to enforce financial and logical invariants (e.g., `extrinsic >= 0`, `Allocated <= Offered`, `payoff - premium == PnL`).
- **Testing & Validation:** Assert every numeric output against expected values through end-to-end phase runs in Jupyter notebooks.
- **Unit Testing Integration:** Supplement the Jupyter end-to-end assertions with standard unit tests using `pytest` to isolate and test individual mathematical, parsing, and pipeline functions.

Data & Temporal Handling

- **Temporal Discipline (D-1):** Enforce an explicit `as_of: date` parameter for all pricing, classification, and hedging functions. Filter inputs using `trade_date <= as_of` as the initial

operation, backed by assertions, to eliminate implicit “today” references.

- **Data Checkpointing:** Implement boundary-based checkpoints using pickled DataFrames to allow independent inspection and re-running of raw, parsed, and validated stages without re-fetching API data.

- **Database Management:** Utilize serverless SQLite databases for local, zero-setup inspectability of output data.

Environments & Reproducibility

- **Reproducibility & Environments:** Use Python 3.12 with `uv` to maintain a single data ecosystem. Pin exact dependency versions using `uv.lock` to ensure clean checkouts run seamlessly with no manual steps beyond `uv sync` and an `.env` file.

- **Version Control & CI/CD:** Mandate conventional commit messages and ensure that `uv sync`, linting, and tests run automatically via pre-commit hooks or continuous integration workflows on every pull request.

Logging & Presentation

- **Logging:** Utilize `logging.getLogger(__name__)` in every module and strictly avoid standard `print` statements. Rely on the `rich` library for consistent logging and progress bars across CLI commands. Restrict `print` or `display` equivalents exclusively to CLI commands and dashboard interfaces.

- **Presentation & UI:** Prioritize clean, polished output — clear formatting, readable tables, and thoughtful layout matter, not just correctness. Employ Plotly for live, interactive charting that renders consistently, and use PySide6 (Qt) for native table views to browse raw data efficiently.